

Traivo Orderkoncept - GAP-Analys v2.0

Session: 2026-06-16T17-42

Datum: 2026-06-16

Executive Summary

Denna uppdaterade GAP-analys baseras på session `2026-06-16T17-42.json` och identifierar kritiska implementationsluckor i Traivo-systemet med fokus på:

1. **Metadatahantering** - Hierarkisk struktur och namnkonventioner
 2. **Villkorstestning** - Saknad validerings- och testfunktion
 3. **Filtreringslogik** - Standardiserad matchningsfunktion
 4. **Artikelhantering** - Ekonomi, leverantörer, strukturartiklar
 5. **Team & Produktionsutrustning** - Komplet register- och kopplingsstruktur
 6. **Ekonomisk uppföljning** - Projekt- och kostnadsställekodifiering
-

1. Sammanfattning av Session 2026-06-16T17-42

1.1 Huvudämnen

Metadatainställningar

- Hierarkisk metadatastruktur ("släktträd") med överordnade och underordnade fält
- Exempel: "kontakt" (överordnat) → "e-post", "namn", "telefonnummer", "titel" (underordnade)
- **Problem:** Underordnade fält presenteras inte med fullständigt namn ("kontaktpost" istället för bara "e-post")
- **Problem:** Sorteringsordning fungerar inte korrekt trots angiven funktion
- Gruppering av metadata: grunduppgifter, produktionsuppgifter

Villkorstestning

- Behov av testfunktion för att validera metadata-villkor
- Ska fungera både i artiklar och orderkoncept
- Användaren ska kunna peka på specifikt objekt/kluster (t.ex. "Axfood Sverige")
- Systemet ska visa hur många träffar villkoret ger

Standardiserad Filterlogik

- **BESLUT:** "Objekttyp"-funktionen ska tas bort (ologisk, använder inaktuell data)
- **NY STANDARD:** Matchningsfunktion med följande logik:
- Välj metadatafält
- Välj operator (lika med / skilt från)
- Plus-knapp för att lägga till fler villkor
- Ska användas konsekvent överallt: artikelfilter, orderkoncept, villkor

Artikelhantering - Ekonomi

- **Prisuppbbyggnad:**
 1. Inköpspris (grundkostnad)
 2. Fraktkostnad/hemtagningskostnad
 3. Lagerkostnad (ränta, hantering, kassation)
 4. Påslag (t.ex. 42% för 30% marginal)
 5. → Listpris (beräknat)
- Marginalberäkning per enhet

Artikelhantering - Fastställnings- och Antalslogik

- **Fast antal:** För besiktningsartiklar (alltid 1)
- **NY FUNKTION:** Kryss för "Ta bort antalsfält i appen"
- När artikeln har fast antal och underliggande information är binär (t.ex. "kroken hel eller trasig")
- Antalsfältet blir redundant och kan döljas

Artikelhantering - Informationshämtning (Metadata)

- Artiklar ska kunna hämta och visa metadata från objekt
- **Tre kryssalternativ per metadatafält:**
 1. ☒ **Visa värde** (presentera befintligt värde)
 2. ☒ **Tillåt uppdatering** (operatör får ändra)
 3. ☒ **Krav för slutförande** (måste uppdateras innan uppgift slutförs)
- Plus-knapp för att lägga till flera metadatafält
- Samma logik som för villkor

Leverantörshantering

- Sidoregister för leverantörer
- Leverantörens artikelnummer kopplas till specifik leverantör
- Möjlig rangordning av leverantörer (prioritet)

Strukturartiklar

- Underartiklar kan läggas till på huvudartiklar
- Antalsstyrning: Följer huvudartikels antal ELLER använder egen logik

Utförarkategori

- Ska matcha mot utförarregistret
- Används för att tilldela rätt utförare till artiklar

Sidoregister

Systemet behöver följande sidoregister:

1. **Artikeltyp** (enkelt register: vara/kontroll/tjänst)
2. **Utförandekoder** (kopplas till artiklar och att-göra-listor)
3. **Leverantörer** (för artikelinköp)
4. **Ikonregister** (bilder för visuell representation)
5. **Metadata-typer** (redan finns, mer komplicerat)

Team, Utförare och Produktionsutrustning

Grundstruktur:

- **Utförare** = Enskilda personer med egen inloggning
- **Team** = 2+ personer med:
- Gemensam inloggning

- Gemensam produktionskontext
- **Produktionsutrustning** = Bilar och verktyg

Register som behövs:

1. Register för människor (utförare)
2. Register för produktionsutrustning (bilar)
3. Möjlighet att gruppera dessa

Kopplingar:

- Team kan ha produktionsutrustning (bilar) kopplade till sig
- Utförandekoder kopplas till team/utrustning
- Geografiska områden kopplas till team/utrustning

Projekt- och Kostnadsställekodifiering

Vid artikelutförande:

När en utförare klarmarkerar en artikel:

1. Uppgiften kopplas till **team**
2. Registrerar vilka **personer** som deltog
3. Registrerar vilken **bil** som användes
4. Fakturering/Fortnox-API använder:
 - **Projektkod** = team/person som utförde
 - **Kostnadsställe** = bil/produktionsutrustning som användes

Ekonomisk uppföljning:

- Artiklar får projekt- och kostnadsställekode
- Exempel: Antal tvättade kärl matchas mot bilens kostnader
- Tankningskostnader:
 - **Kostnadsställe** (vilken bil)
 - **Projektnummer** (vem som tankade)
- Möjliggör detaljerad ekonomisk analys per bil/team

Menyförenkling

System ska fokusera på följande huvudfunktioner:

1. Tanka in och uppdatera objekt
2. Skapa orderkoncept
3. Skapa och hantera artiklar
4. Hantera metadata
5. Lägga upp separata prislistor för kunder
6. (Utförartidsregister per utförare)

2. Jämförelse mot Tidigare GAP-Analys

2.1 Bekräftade GAP från Tidigare Session

Följande GAP från tidigare session 2026-06-16T16:56 bekräftas och förtydligas:

GAP från v1	Status i v2	Kommentar
Saknad "order_concepts" datamodell	Bekräftat	Ingen ny information om implementationsstatus
Ingen villkorsmotor	Utökad kravbild	Nu med krav på testfunktion och standardiserad matchningslogik
Saknad objekthierarki-hantering	Bekräftat	Metadata-hierarki ("släkt-träd") behöver implementeras
Ingen faktureringslogik	Kraftigt utökad	Nu med projekt-/kostnadsställekodifiering och Fortnox-integration
Saknad wizard-komponent	Bekräftat	Ingen ny information

2.2 Nya Insikter och Förtydliganden

Metadata

- **Tidigare:** Vagt beskrivet behov av metadata-hantering
- **Nu:** Detaljerad kravbild med hierarkisk struktur, namnkonventioner, sortering och testning

Artikelekonomi

- **Tidigare:** Grundläggande prissättning
- **Nu:** Komplet kostnadsuppbyggnad med inköpspris → frakt → lager → påslag → listpris

Team och Produktionsutrustning

- **Tidigare:** Inte omnämnt
- **Nu: HELT NYTT** - Omfattande krav på team-struktur, produktionsutrustning och ekonomisk uppföljning

Fortnox-integration

- **Tidigare:** Inte omnämnt
- **Nu: HELT NYTT** - Projekt- och kostnadsställekodifiering för API-integration

3. NYA GAP Identifierade i Session v2

3.1 KRITISKA GAP (Blockerar kärnfunktionalitet)

GAP-101: Hierarkisk Metadatastruktur

Område: Backend - Datamodell

Prioritet: ● KRITISK

Beskrivning:

- Metadata måste stödja hierarki (parent-child relationer)

- Underordnade fält ska presenteras med sammansatt namn (parent + child)
- Exempel: "kontakt" + "e-post" = "kontaktepost"

Teknisk Implementation:

```
// packages/shared/src/types/metadata.ts
export interface MetadataField {
  id: string;
  name: string;
  displayName: string; // Sammansatt namn för underordnade fält
  type: 'text' | 'number' | 'date' | 'select' | 'multiselect' | 'boolean';
  parentId: string | null; // Referens till överordnat fält
  sortOrder: number;
  group: 'basic_info' | 'production_info' | 'custom';
  isActive: boolean;
  allowedValues?: string[]; // För select/multiselect
}

export interface MetadataHierarchy {
  field: MetadataField;
  children: MetadataField[];
  fullPath: string; // "kontakt.e-post"
  fullDisplayName: string; // "kontaktepost"
}
```

Databas-schema (Drizzle ORM):

```
// packages/db/src/schema/metadata.ts
export const metadataFields = pgTable('metadata_fields', {
  id: uuid('id').primaryKey().defaultRandom(),
  name: varchar('name', { length: 255 }).notNull(),
  displayName: varchar('display_name', { length: 255 }).notNull(),
  type: varchar('type', { length: 50 }).notNull(),
  parentId: uuid('parent_id').references(() => metadataFields.id, { onDelete: 'cascade' }),
  sortOrder: integer('sort_order').default(0),
  group: varchar('group', { length: 50 }).notNull(),
  isActive: boolean('is_active').default(true),
  allowedValues: jsonb('allowed_values'),
  createdAt: timestamp('created_at').defaultNow(),
  updatedAt: timestamp('updated_at').defaultNow(),
});

export const metadataFieldsRelations = relations(metadataFields, ({ one, many }) => ({
  parent: one(metadataFields, {
    fields: [metadataFields.parentId],
    references: [metadataFields.id],
  }),
  children: many(metadataFields),
}));
```

API Endpoints:

- GET /api/metadata/fields?includeHierarchy=true - Hämta alla fält med hierarki
- GET /api/metadata/fields/:id/hierarchy - Hämta fält med alla parents och children
- PATCH /api/metadata/fields/:id/sort-order - Uppdatera sorteringsordning
- POST /api/metadata/fields/:id/children - Skapa underordnat fält

Beroenden: GAP-102, GAP-201

GAP-102: Villkorstestning och Validering

Område: Backend + Frontend

Prioritet:  KRITISK

Beskrivning:

- Testfunktion för att validera metadata-villkor
- Ska fungera både för artiklar och orderkoncept
- Användaren pekar på objekt/kluster för att testa
- Systemet visar antal träffar

Teknisk Implementation:

Backend - Villkorsmotor:

```

// packages/api/src/services/condition-engine.ts
export interface Condition {
  id: string;
  metadataFieldId: string;
  operator: 'equals' | 'not_equals' | 'contains' | 'not_contains' | 'greater_than' | 'less_than';
  value: any;
  logicalOperator?: 'AND' | 'OR'; // För nästa villkor
}

export interface ConditionGroup {
  conditions: Condition[];
  type: 'article' | 'order_concept';
  targetId: string; // artikel-id eller orderkoncept-id
}

export class ConditionEngine {
  // Testa villkor mot specifikt objekt/kluster
  async testConditions(
    conditions: Condition[],
    testContext: {
      objectId?: string;
      clusterId?: string;
      customMetadata?: Record<string, any>;
    }
  ): Promise<{
    matches: boolean;
    matchCount: number;
    failedConditions: Condition[];
    matchedObjects: Array<{ id: string; name: string }>;
  }> {
    // Implementation
  }

  // Evaluera ett enskilt villkor
  private evaluateCondition(
    condition: Condition,
    metadata: Record<string, any>
  ): boolean {
    const fieldValue = metadata[condition.metadataFieldId];

    switch (condition.operator) {
      case 'equals':
        return fieldValue === condition.value;
      case 'not_equals':
        return fieldValue !== condition.value;
      case 'contains':
        return String(fieldValue).includes(String(condition.value));
      case 'not_contains':
        return !String(fieldValue).includes(String(condition.value));
      case 'greater_than':
        return Number(fieldValue) > Number(condition.value);
      case 'less_than':
        return Number(fieldValue) < Number(condition.value);
      default:
        return false;
    }
  }

  // Evaluera grupp av villkor med logiska operatorer
  private evaluateConditionGroup(
    conditions: Condition[],

```

```

    metadata: Record<string, any>
  ): boolean {
    if (conditions.length === 0) return true;

    let result = this.evaluateCondition(conditions[0], metadata);

    for (let i = 1; i < conditions.length; i++) {
      const currentResult = this.evaluateCondition(conditions[i], metadata);
      const operator = conditions[i - 1].logicalOperator || 'AND';

      if (operator === 'AND') {
        result = result && currentResult;
      } else {
        result = result || currentResult;
      }
    }

    return result;
  }
}

```

Databas-schema:

```

// packages/db/src/schema/conditions.ts
export const conditions = pgTable('conditions', {
  id: uuid('id').primaryKey().defaultRandom(),
  metadataFieldId: uuid('metadata_field_id').references(() => metadataFields.id).notNull(),
  operator: varchar('operator', { length: 50 }).notNull(),
  value: jsonb('value').notNull(),
  logicalOperator: varchar('logical_operator', { length: 10 }), // 'AND' | 'OR'
  sortOrder: integer('sort_order').default(0),
  createdAt: timestamp('created_at').defaultNow(),
});

export const articleConditions = pgTable('article_conditions', {
  id: uuid('id').primaryKey().defaultRandom(),
  articleId: uuid('article_id').references(() => articles.id, { onDelete: 'cascade' }).notNull(),
  conditionId: uuid('condition_id').references(() => conditions.id, { onDelete: 'cascade' }).notNull(),
  createdAt: timestamp('created_at').defaultNow(),
});

export const orderConceptConditions = pgTable('order_concept_conditions', {
  id: uuid('id').primaryKey().defaultRandom(),
  orderConceptId: uuid('order_concept_id').references(() => orderConcepts.id, { onDelete: 'cascade' }).notNull(),
  conditionId: uuid('condition_id').references(() => conditions.id, { onDelete: 'cascade' }).notNull(),
  createdAt: timestamp('created_at').defaultNow(),
});

```

API Endpoints:

- POST /api/conditions/test - Testa villkor mot testdata
- GET /api/conditions/:id/evaluate - Evaluera specifikt villkor
- POST /api/articles/:id/conditions/test - Testa artikelvillkor
- POST /api/order-concepts/:id/conditions/test - Testa orderkonceptvillkor

Frontend - Testkomponent:

```
// apps/mobile/src/components/conditions/ConditionTester.tsx
import React, { useState } from 'react';
import { View, Text, Button } from 'react-native';

interface ConditionTesterProps {
  conditions: Condition[];
  type: 'article' | 'order_concept';
}

export const ConditionTester: React.FC<ConditionTesterProps> = ({
  conditions,
  type,
}) => {
  const [testResult, setTestResult] = useState<{
    matches: boolean;
    matchCount: number;
    matchedObjects: Array<{ id: string; name: string }>;
  } | null>(null);

  const [selectedObject, setSelectedObject] = useState<string>('');

  const handleTest = async () => {
    const result = await api.conditions.test({
      conditions,
      testContext: {
        objectId: selectedObject || undefined,
      },
    });
    setTestResult(result);
  };

  return (
    <View>
      <Text>Testa villkor</Text>
      <ObjectPicker
        placeholder="Välj objekt/kluster (t.ex. Axfood Sverige)"
        onSelect={setSelectedObject}
      />
      <Button title="Testa" onPress={handleTest} />

      {testResult && (
        <View>
          <Text>Resultat: {testResult.matches ? '✓ Matchar' : 'x Matchar inte'}</Text>
          <Text>Antal träffar: {testResult.matchCount}</Text>
          {testResult.matchedObjects.length > 0 && (
            <View>
              <Text>Matchade objekt:</Text>
              {testResult.matchedObjects.map(obj => (
                <Text key={obj.id}>- {obj.name}</Text>
              ))}
            </View>
          )}
        </View>
      )}
    </View>
  );
};
```

GAP-103: Standardiserad Matchningsfunktion (Filter)

Område: Frontend + Backend

Prioritet: ● KRITISK

Beskrivning:

- Ta bort "Objekttyp"-funktionen (ologisk, inaktuell data)
- Implementera standardiserad matchningsfunktion
- Används överallt: artikelfilter, orderkoncept, villkor
- Logik: Välj metadatafält → operator → värde → plus för att lägga till fler

Teknisk Implementation:

Återanvändbar Filterkomponent:

```
// apps/mobile/src/components/filters/MetadataFilter.tsx
import React, { useState } from 'react';
import { View, Button } from 'react-native';
import { Picker } from '@react-native-picker/picker';

interface FilterRule {
  id: string;
  metadataFieldId: string;
  operator: 'equals' | 'not_equals';
  value: any;
}

interface MetadataFilterProps {
  onFilterChange: (filters: FilterRule[]) => void;
  availableFields: MetadataField[];
}

export const MetadataFilter: React.FC<MetadataFilterProps> = ({
  onFilterChange,
  availableFields,
}) => {
  const [filters, setFilters] = useState<FilterRule[]>([
    { id: '1', metadataFieldId: '', operator: 'equals', value: '' }
  ]);

  const addFilter = () => {
    setFilters([
      ...filters,
      { id: Date.now().toString(), metadataFieldId: '', operator: 'equals', value: '' }
    ]);
  };

  const updateFilter = (id: string, updates: Partial<FilterRule>) => {
    const updated = filters.map(f =>
      f.id === id ? { ...f, ...updates } : f
    );
    setFilters(updated);
    onFilterChange(updated);
  };

  const removeFilter = (id: string) => {
    const updated = filters.filter(f => f.id !== id);
    setFilters(updated);
    onFilterChange(updated);
  };

  return (
    <View>
      {filters.map((filter, index) => (
        <View key={filter.id}>
          {/* Välj metadatafält */}
          <Picker
            selectedValue={filter.metadataFieldId}
            onValueChange={(value) => updateFilter(filter.id, { metadataFieldId:
value }}}
          >
            <Picker.Item label="Välj fält..." value="" />
            {availableFields.map(field => (
              <Picker.Item
                key={field.id}
                label={field.displayName}

```

```

        value={field.id}
      />
    ))}
  </Picker>

  { /* Välj operator */ }
  <Picker
    selectedValue={filter.operator}
    onChange={value => updateFilter(filter.id, { operator: value })}
  >
    <Picker.Item label="Lika med" value="equals" />
    <Picker.Item label="Skilt från" value="not_equals" />
  </Picker>

  { /* Värdefält - anpassas efter fälttyp */ }
  <MetadataValueInput
    metadataField={availableFields.find(f => f.id === filter.metadataFieldId)}
    value={filter.value}
    onChange={value => updateFilter(filter.id, { value })}
  />

  { /* Ta bort filter */ }
  { filters.length > 1 && (
    <Button title="Ta bort" onPress={() => removeFilter(filter.id)} />
  ) }
  </View>
)}

{ /* Lägg till nytt filter */ }
<Button title="+ Lägg till filter" onPress={addFilter} />
</View>
);
};

```

Backend - Filterlogik:

```
// packages/api/src/services/filter.service.ts
export interface FilterRule {
  metadataFieldId: string;
  operator: 'equals' | 'not_equals';
  value: any;
}

export class FilterService {
  // Applicera filter på objekt-query
  async applyFilters(
    baseQuery: any, // Drizzle query
    filters: FilterRule[]
  ): Promise<any> {
    let query = baseQuery;

    for (const filter of filters) {
      const field = await db.query.metadataFields.findFirst({
        where: eq(metadataFields.id, filter.metadataFieldId),
      });

      if (!field) continue;

      // Applicera filter baserat på operator
      if (filter.operator === 'equals') {
        query = query.where(
          sql`metadata->'${field.name}' = ${JSON.stringify(filter.value)}`
        );
      } else if (filter.operator === 'not_equals') {
        query = query.where(
          sql`metadata->'${field.name}' != ${JSON.stringify(filter.value)}`
        );
      }
    }

    return query;
  }
}
```

API Endpoints:

- POST /api/articles/filter - Filtrera artiklar
- POST /api/objects/filter - Filtrera objekt
- POST /api/order-concepts/filter - Filtrera orderkoncept

Migration för borttagning av "Objekttyp":

```
// packages/db/migrations/XXXX_remove_object_type.ts
export async function up(db: Database) {
  // Ta bort objekttyp-fält från artiklar
  await db.schema.alterTable('articles', (table) => {
    table.dropColumn('object_type');
  });

  // Ta bort objekttyp-register om det finns
  await db.schema.dropTableIfExists('object_types');
}
```

Beroenden: GAP-101, GAP-102

GAP-104: Artikel - Ekonomisk Prisuppbyggnad

Område: Backend - Datamodell + Beräkningslogik

Prioritet: 🟡 HÖG

Beskrivning:

Artikelekonomi ska stödja komplett kostnadsuppbyggnad:

1. Inköpspris
2. Fraktkostnad/hemtagningskostnad
3. Lagerkostnad (ränta, hantering, kassation)
4. Påslag (procent)
5. → Beräknat listpris
6. → Beräknad marginal

Teknisk Implementation:

Databas-schema:

```
// packages/db/src/schema/articles.ts (utökning)
export const articles = pgTable('articles', {
  // ... befintliga fält ...

  // Ekonomi
  purchasePrice: decimal('purchase_price', { precision: 10, scale: 2 }), // Inköpspris
  freightCost: decimal('freight_cost', { precision: 10, scale: 2 }), // Fraktkostnad
  inventoryCost: decimal('inventory_cost', { precision: 10, scale: 2 }), // Lagerkost-
  nad
  markup: decimal('markup', { precision: 5, scale: 2 }), // Påslag (procent, t.ex.
  42.00)
  listPrice: decimal('list_price', { precision: 10, scale: 2 }), // Listpris (ber-
  äknat)
  calculatedMargin: decimal('calculated_margin', { precision: 5, scale: 2 }), // Mar-
  ginal (procent)
  marginPerUnit: decimal('margin_per_unit', { precision: 10, scale: 2 }), // Marginal
  per enhet (SEK)

  // ... övriga fält ...
});
```

Backend - Prisberäkning:

```
// packages/api/src/services/article-pricing.service.ts
export interface ArticlePricing {
  purchasePrice: number;
  freightCost: number;
  inventoryCost: number;
  markup: number; // Procent
}

export class ArticlePricingService {
  // Beräkna listpris
  calculateListPrice(pricing: ArticlePricing): number {
    const totalCost =
      pricing.purchasePrice +
      pricing.freightCost +
      pricing.inventoryCost;

    const listPrice = totalCost * (1 + pricing.markup / 100);
    return Math.round(listPrice * 100) / 100; // Runda till 2 decimaler
  }

  // Beräkna marginal (procent)
  calculateMarginPercent(pricing: ArticlePricing, listPrice: number): number {
    const totalCost =
      pricing.purchasePrice +
      pricing.freightCost +
      pricing.inventoryCost;

    const margin = ((listPrice - totalCost) / listPrice) * 100;
    return Math.round(margin * 100) / 100;
  }

  // Beräkna marginal per enhet (SEK)
  calculateMarginPerUnit(pricing: ArticlePricing, listPrice: number): number {
    const totalCost =
      pricing.purchasePrice +
      pricing.freightCost +
      pricing.inventoryCost;

    return Math.round((listPrice - totalCost) * 100) / 100;
  }

  // Uppdatera alla beräknade värden
  async updateArticlePricing(articleId: string, pricing: ArticlePricing):
  Promise<void> {
    const listPrice = this.calculateListPrice(pricing);
    const marginPercent = this.calculateMarginPercent(pricing, listPrice);
    const marginPerUnit = this.calculateMarginPerUnit(pricing, listPrice);

    await db.update(articles)
      .set({
        purchasePrice: pricing.purchasePrice.toString(),
        freightCost: pricing.freightCost.toString(),
        inventoryCost: pricing.inventoryCost.toString(),
        markup: pricing.markup.toString(),
        listPrice: listPrice.toString(),
        calculatedMargin: marginPercent.toString(),
        marginPerUnit: marginPerUnit.toString(),
        updatedAt: new Date(),
      })
      .where(eq(articles.id, articleId));
  }
}
```

Frontend - Prisberäkningsformulär:


```
// apps/mobile/src/screens/articles/ArticlePricingForm.tsx
import React, { useState, useEffect } from 'react';
import { View, Text, TextInput } from 'react-native';

export const ArticlePricingForm = () => {
  const [purchasePrice, setPurchasePrice] = useState<number>(0);
  const [freightCost, setFreightCost] = useState<number>(0);
  const [inventoryCost, setInventoryCost] = useState<number>(0);
  const [markup, setMarkup] = useState<number>(0);

  const [listPrice, setListPrice] = useState<number>(0);
  const [marginPercent, setMarginPercent] = useState<number>(0);
  const [marginPerUnit, setMarginPerUnit] = useState<number>(0);

  // Beräkna automatiskt när värden ändras
  useEffect(() => {
    const totalCost = purchasePrice + freightCost + inventoryCost;
    const calculated = totalCost * (1 + markup / 100);
    setListPrice(calculated);

    const margin = ((calculated - totalCost) / calculated) * 100;
    setMarginPercent(margin);

    setMarginPerUnit(calculated - totalCost);
  }, [purchasePrice, freightCost, inventoryCost, markup]);

  return (
    <View>
      <Text>Inköpspris:</Text>
      <TextInput
        keyboardType="numeric"
        value={purchasePrice.toString()}
        onChangeText={(val) => setPurchasePrice(Number(val))}
      />

      <Text>Fraktkostnad:</Text>
      <TextInput
        keyboardType="numeric"
        value={freightCost.toString()}
        onChangeText={(val) => setFreightCost(Number(val))}
      />

      <Text>Lagerkostnad:</Text>
      <TextInput
        keyboardType="numeric"
        value={inventoryCost.toString()}
        onChangeText={(val) => setInventoryCost(Number(val))}
      />

      <Text>Påslag (%):</Text>
      <TextInput
        keyboardType="numeric"
        value={markup.toString()}
        onChangeText={(val) => setMarkup(Number(val))}
      />

      <View style={{ marginTop: 20, padding: 10, backgroundColor: '#f0f0f0' }}>
        <Text style={{ fontWeight: 'bold' }}>Beräknat listpris:
        {listPrice.toFixed(2)} SEK</Text>
        <Text>Marginal: {marginPercent.toFixed(2)}%</Text>
        <Text>Marginal per enhet: {marginPerUnit.toFixed(2)} SEK</Text>
      </View>
    </View>
  );
}
```

```
    </View>  
  );  
};
```

API Endpoints:

- PUT /api/articles/:id/pricing - Uppdatera artikelpriser
- POST /api/articles/pricing/calculate - Beräkna priser (preview utan att spara)

Beroenden: Ingen

GAP-105: Artikel - Informationshämtning från Metadata

Område: Backend + Frontend

Prioritet: 🟡 HÖG

Beskrivning:

Artiklar ska kunna hämta och visa metadata från objekt med tre beteenden per fält:

1. ☒ Visa värde (presentera befintligt värde)
2. ☒ Tillåt uppdatering (operatör får ändra)
3. ☒ Krav för slutförande (måste uppdateras)

Teknisk Implementation:

Databas-schema:

```
// packages/db/src/schema/articles.ts (ny tabell)
export const articleMetadataBindings = pgTable('article_metadata_bindings', {
  id: uuid('id').primaryKey().defaultRandom(),
  articleId: uuid('article_id').references(() => articles.id, { onDelete: 'cascade' })
  .notNull(),
  metadataFieldId: uuid('metadata_field_id').references(() => metadataFields.id).notNull(),

  // Beteenden
  showValue: boolean('show_value').default(true), // Visa befintligt värde
  allowUpdate: boolean('allow_update').default(false), // Operatör får ändra
  requiredForCompletion: boolean('required_for_completion').default(false), // Måste uppdateras

  sortOrder: integer('sort_order').default(0),
  createdAt: timestamp('created_at').defaultNow(),
  updatedAt: timestamp('updated_at').defaultNow(),
});

export const articleMetadataBindingsRelations = relations(
  articleMetadataBindings,
  ({ one }) => ({
    article: one(articles, {
      fields: [articleMetadataBindings.articleId],
      references: [articles.id],
    }),
    metadataField: one(metadataFields, {
      fields: [articleMetadataBindings.metadataFieldId],
      references: [metadataFields.id],
    }),
  })
);
```

Backend - Hämta metadata för artikel:

```
// packages/api/src/services/article-metadata.service.ts
export interface ArticleMetadataBinding {
  metadataField: MetadataField;
  currentValue: any;
  showValue: boolean;
  allowUpdate: boolean;
  requiredForCompletion: boolean;
}

export class ArticleMetadataService {
  // Hämta alla metadata-bindningar för artikel i kontext av ett objekt
  async getArticleMetadata(
    articleId: string,
    objectId: string
  ): Promise<ArticleMetadataBinding[]> {
    // Hämta bindningar
    const bindings = await db.query.articleMetadataBindings.findMany({
      where: eq(articleMetadataBindings.articleId, articleId),
      with: {
        metadataField: true,
      },
      orderBy: [asc(articleMetadataBindings.sortOrder)],
    });

    // Hämta objekt-metadata
    const object = await db.query.objects.findFirst({
      where: eq(objects.id, objectId),
    });

    // Kombinera bindningar med faktiska värden
    return bindings.map(binding => ({
      metadataField: binding.metadataField,
      currentValue: object?.metadata?.[binding.metadataField.name] || null,
      showValue: binding.showValue,
      allowUpdate: binding.allowUpdate,
      requiredForCompletion: binding.requiredForCompletion,
    }));
  }

  // Validera att alla required-fält är ifyllda
  async validateRequiredMetadata(
    articleId: string,
    objectId: string
  ): Promise<{ valid: boolean; missingFields: string[] }> {
    const metadata = await this.getArticleMetadata(articleId, objectId);

    const missingFields = metadata
      .filter(m => m.requiredForCompletion && !m.currentValue)
      .map(m => m.metadataField.displayName);

    return {
      valid: missingFields.length === 0,
      missingFields,
    };
  }
}
```

Frontend - Metadata-visning i uppgift:

```
// apps/mobile/src/components/tasks/TaskMetadataFields.tsx
import React from 'react';
import { View, Text, TextInput } from 'react-native';

interface TaskMetadataFieldsProps {
  articleId: string;
  objectId: string;
  onMetadataUpdate: (fieldId: string, value: any) => void;
}

export const TaskMetadataFields: React.FC<TaskMetadataFieldsProps> = ({
  articleId,
  objectId,
  onMetadataUpdate,
}) => {
  const { data: metadataBindings } = useQuery(
    ['article-metadata', articleId, objectId],
    () => api.articles.getMetadata(articleId, objectId)
  );

  if (!metadataBindings) return null;

  return (
    <View>
      <Text style={{ fontWeight: 'bold', marginBottom: 10 }}>
        Objekt-information
      </Text>

      {metadataBindings.map(binding => {
        if (!binding.showValue) return null;

        return (
          <View key={binding.metadataField.id} style={{ marginBottom: 15 }}>
            <Text>{binding.metadataField.displayName}</Text>

            {binding.allowUpdate ? (
              // Redigerbart fält
              <MetadataInput
                field={binding.metadataField}
                value={binding.currentValue}
                onChange={(value) => onMetadataUpdate(binding.metadataField.id, value)}
              />
            ) : (
              // Endast läsbart
              <Text style={{ color: '#666' }}>
                {binding.currentValue || '(Inget värde)'}
              </Text>
            )}

            {binding.requiredForCompletion && (
              <Text style={{ color: 'red', fontSize: 12 }}>
                * Krävs för att slutföra
              </Text>
            )}
          </View>
        );
      })}
    </View>
  );
};
```

API Endpoints:

- GET /api/articles/:id/metadata-bindings - Hämta metadata-bindningar
- POST /api/articles/:id/metadata-bindings - Lägg till metadata-bindning
- PUT /api/articles/:id/metadata-bindings/:bindingId - Uppdatera bindning
- DELETE /api/articles/:id/metadata-bindings/:bindingId - Ta bort bindning
- GET /api/articles/:id/metadata/:objectId - Hämta metadata i kontext av objekt
- POST /api/articles/:id/metadata/:objectId/validate - Validera required-fält

Beroenden: GAP-101

GAP-106: Fast Antal och Dold Antalsinmatning

Område: Backend + Frontend

Prioritet: 🟡 MEDIUM

Beskrivning:

- Besiktningsartiklar har alltid fast antal = 1
- Option att dölja antalsfält i appen när fast antal används
- Minskar friktionen i gränssnittet

Teknisk Implementation:**Databas-schema:**

```
// packages/db/src/schema/articles.ts (utökning)
export const articles = pgTable('articles', {
  // ... befintliga fält ...

  // Antalslogik
  quantityLogic: varchar('quantity_logic', { length: 50 }).default('variable'), //
  'fixed' | 'variable'
  fixedQuantity: integer('fixed_quantity'), // Om fixed: vilket antal?
  hideQuantityInput: boolean('hide_quantity_input').default(false), // Dölj fält i app

  // ... övriga fält ...
});
```

Backend - Validering:

```
// packages/api/src/services/article.service.ts
export class ArticleService {
  async updateQuantityLogic(
    articleId: string,
    logic: 'fixed' | 'variable',
    fixedQuantity?: number,
    hideInput?: boolean
  ): Promise<void> {
    // Validering
    if (logic === 'fixed' && !fixedQuantity) {
      throw new Error('Fixed quantity must be specified when logic is "fixed"');
    }

    await db.update(articles)
      .set({
        quantityLogic: logic,
        fixedQuantity: logic === 'fixed' ? fixedQuantity : null,
        hideQuantityInput: hideInput || false,
        updatedAt: new Date(),
      })
      .where(eq(articles.id, articleId));
  }
}
```

Frontend - Artikelformulär:

```
// apps/mobile/src/screens/articles/ArticleQuantitySettings.tsx
import React, { useState } from 'react';
import { View, Text, Switch, TextInput } from 'react-native';
import { Picker } from '@react-native-picker/picker';

export const ArticleQuantitySettings = () => {
  const [quantityLogic, setQuantityLogic] = useState<'fixed' | 'variable'>('variable')
;
  const [fixedQuantity, setFixedQuantity] = useState<number>(1);
  const [hideInput, setHideInput] = useState<boolean>(false);

  return (
    <View>
      <Text>Antalslogik:</Text>
      <Picker
        selectedValue={quantityLogic}
        onValueChange={setQuantityLogic}
      >
        <Picker.Item label="Variabel" value="variable" />
        <Picker.Item label="Fast antal" value="fixed" />
      </Picker>

      {quantityLogic === 'fixed' && (
        <>
          <Text>Fast antal:</Text>
          <TextInput
            keyboardType="numeric"
            value={fixedQuantity.toString()}
            onChangeText={(val) => setFixedQuantity(Number(val))}
          />

          <View style={{ flexDirection: 'row', alignItems: 'center', marginTop: 10 }}>
            <Switch
              value={hideInput}
              onValueChange={setHideInput}
            />
            <Text style={{ marginLeft: 10 }}>
              Dölj antalsfält i appen (rekommenderas för besiktningsartiklar)
            </Text>
          </View>
        </>
      )}
    </View>
  );
};
```

Frontend - Uppgiftsvisning:


```
// apps/mobile/src/components/tasks/TaskQuantityInput.tsx
import React from 'react';
import { View, Text, TextInput } from 'react-native';

interface TaskQuantityInputProps {
  article: Article;
  quantity: number;
  onQuantityChange: (quantity: number) => void;
}

export const TaskQuantityInput: React.FC<TaskQuantityInputProps> = ({
  article,
  quantity,
  onQuantityChange,
}) => {
  // Om fast antal och dold input, visa inget
  if (article.quantityLogic === 'fixed' && article.hideQuantityInput) {
    return null;
  }

  // Om fast antal men inte dold, visa värdet (ej redigerbart)
  if (article.quantityLogic === 'fixed') {
    return (
      <View>
        <Text>Antal: {article.fixedQuantity}</Text>
      </View>
    );
  }

  // Variabel, visa input
  return (
    <View>
      <Text>Antal:</Text>
      <TextInput
        keyboardType="numeric"
        value={quantity.toString()}
        onChangeText={(val) => onQuantityChange(Number(val))}
      />
    </View>
  );
};
```

Beroenden: Ingen

GAP-107: Strukturartiklar (Underartiklar)

Område: Backend - Datamodell

Prioritet: 🟡 MEDIUM

Beskrivning:

- Artiklar kan ha underartiklar (strukturartiklar)
- Underartiklar kan följa huvudartikels antal eller ha egen logik

Teknisk Implementation:

Databas-schema:

```
// packages/db/src/schema/articles.ts (ny tabell)
export const articleStructures = pgTable('article_structures', {
  id: uuid('id').primaryKey().defaultRandom(),
  parentArticleId: uuid('parent_article_id').references(() => articles.id, {
    onDelete: 'cascade' }).notNull(),
  childArticleId: uuid('child_article_id').references(() => articles.id, { onDelete: '
  cascade' }).notNull(),

  // Antalslogik för underartikel
  quantityBehavior: varchar('quantity_behavior', { length: 50 }).default('fol-
  low_parent'),
  // 'follow_parent' = Följer huvudartikels antal
  // 'use_own_logic' = Använder egen artikel-logik

  multiplifier: decimal('multiplier', { precision: 10, scale: 2 }).default('1'), // Om
  follow_parent: multiplicera med detta
  sortOrder: integer('sort_order').default(0),
  createdAt: timestamp('created_at').defaultNow(),
});

export const articleStructuresRelations = relations(articleStructures, ({ one }) => ({
  parentArticle: one(articles, {
    fields: [articleStructures.parentArticleId],
    references: [articles.id],
    relationName: 'parent',
  }),
  childArticle: one(articles, {
    fields: [articleStructures.childArticleId],
    references: [articles.id],
    relationName: 'child',
  }),
}));
```

Backend - Strukturartikel-hantering:

```
// packages/api/src/services/article-structure.service.ts
export class ArticleStructureService {
  // Hämta alla underartiklar för en artikel
  async getChildArticles(parentArticleId: string): Promise<Array<{
    article: Article;
    quantityBehavior: string;
    multiplier: number;
  }>> {
    const structures = await db.query.articleStructures.findMany({
      where: eq(articleStructures.parentArticleId, parentArticleId),
      with: {
        childArticle: true,
      },
      orderBy: [asc(articleStructures.sortOrder)],
    });

    return structures.map(s => ({
      article: s.childArticle,
      quantityBehavior: s.quantityBehavior,
      multiplier: Number(s.multiplier),
    }));
  }

  // Beräkna antal för underartikel
  calculateChildQuantity(
    parentQuantity: number,
    childArticle: Article,
    quantityBehavior: string,
    multiplier: number
  ): number {
    if (quantityBehavior === 'follow_parent') {
      return parentQuantity * multiplier;
    } else {
      // Använd artikelns egen logik
      return childArticle.fixedQuantity || 1;
    }
  }

  // Lägg till underartikel
  async addChildArticle(
    parentArticleId: string,
    childArticleId: string,
    quantityBehavior: 'follow_parent' | 'use_own_logic',
    multiplier: number = 1
  ): Promise<void> {
    await db.insert(articleStructures).values({
      parentArticleId,
      childArticleId,
      quantityBehavior,
      multiplier: multiplier.toString(),
    });
  }
}
```

API Endpoints:

- GET /api/articles/:id/children - Hämta underartiklar
- POST /api/articles/:id/children - Lägg till underartikel
- PUT /api/articles/:parentId/children/:childId - Uppdatera relation
- DELETE /api/articles/:parentId/children/:childId - Ta bort underartikel

Beroenden: Ingen

3.2 HÖGA GAP (Kritiskt för affärslogik)

GAP-201: Team, Utförare och Produktionsutrustning

Område: Backend - Komplet registrersystem

Prioritet: 🟡 HÖG

Beskrivning:

Komplett system för:

- **Utförare** (personer med egen inloggning)
- **Team** (grupp av utförare med gemensam inloggning)
- **Produktionsutrustning** (bilar, verktyg)
- Kopplingar mellan dessa
- Utförandekoder och geografiska områden

Teknisk Implementation:

Databas-schema:

```
// packages/db/src/schema/team.ts

// 1. Utförare (Performer)
export const performers = pgTable('performers', {
  id: uuid('id').primaryKey().defaultRandom(),
  userId: uuid('user_id').references(() => users.id, { onDelete: 'cascade' }).notNull(), // Koppling till användare
  firstName: varchar('first_name', { length: 255 }).notNull(),
  lastName: varchar('last_name', { length: 255 }).notNull(),
  email: varchar('email', { length: 255 }).notNull().unique(),
  phone: varchar('phone', { length: 50 }),
  isActive: boolean('is_active').default(true),
  createdAt: timestamp('created_at').defaultNow(),
  updatedAt: timestamp('updated_at').defaultNow(),
});

// 2. Team
export const teams = pgTable('teams', {
  id: uuid('id').primaryKey().defaultRandom(),
  name: varchar('name', { length: 255 }).notNull(),
  description: text('description'),
  sharedUserId: uuid('shared_user_id').references(() => users.id), // Gemensam inloggning
  isActive: boolean('is_active').default(true),
  createdAt: timestamp('created_at').defaultNow(),
  updatedAt: timestamp('updated_at').defaultNow(),
});

// 3. Team-medlemskap
export const teamMembers = pgTable('team_members', {
  id: uuid('id').primaryKey().defaultRandom(),
  teamId: uuid('team_id').references(() => teams.id, { onDelete: 'cascade' }).notNull(),
  performerId: uuid('performer_id').references(() => performers.id, { onDelete: 'cascade' }).notNull(),
  role: varchar('role', { length: 50 }).default('member'), // 'leader' | 'member'
  joinedAt: timestamp('joined_at').defaultNow(),
});

// 4. Produktionsutrustning (Equipment)
export const equipment = pgTable('equipment', {
  id: uuid('id').primaryKey().defaultRandom(),
  type: varchar('type', { length: 50 }).notNull(), // 'vehicle' | 'tool' | 'other'
  name: varchar('name', { length: 255 }).notNull(),
  identifier: varchar('identifier', { length: 100 }), // Regnummer, serienummer
  costCenter: varchar('cost_center', { length: 100 }), // Kostnadsställe
  isActive: boolean('is_active').default(true),
  metadata: jsonb('metadata'), // Flexibel data
  createdAt: timestamp('created_at').defaultNow(),
  updatedAt: timestamp('updated_at').defaultNow(),
});

// 5. Team-utrustning (koppling)
export const teamEquipment = pgTable('team_equipment', {
  id: uuid('id').primaryKey().defaultRandom(),
  teamId: uuid('team_id').references(() => teams.id, { onDelete: 'cascade' }).notNull(),
  equipmentId: uuid('equipment_id').references(() => equipment.id, { onDelete: 'cascade' }).notNull(),
  assignedAt: timestamp('assigned_at').defaultNow(),
});
```

```
// 6. Utförandekoder (Performance Codes)
export const performanceCodes = pgTable('performance_codes', {
  id: uuid('id').primaryKey().defaultRandom(),
  code: varchar('code', { length: 50 }).notNull().unique(),
  name: varchar('name', { length: 255 }).notNull(),
  description: text('description'),
  isActive: boolean('is_active').default(true),
  createdAt: timestamp('created_at').defaultNow(),
});

// 7. Team-utförandekoder (koppling)
export const teamPerformanceCodes = pgTable('team_performance_codes', {
  id: uuid('id').primaryKey().defaultRandom(),
  teamId: uuid('team_id').references(() => teams.id, { onDelete: 'cascade' }).notNull(),
  performanceCodeId: uuid('performance_code_id').references(() => performanceCodes.id, { onDelete: 'cascade' }).notNull(),
});

// 8. Geografiska områden
export const geographicAreas = pgTable('geographic_areas', {
  id: uuid('id').primaryKey().defaultRandom(),
  name: varchar('name', { length: 255 }).notNull(),
  type: varchar('type', { length: 50 }), // 'region' | 'city' | 'postal_code'
  identifier: varchar('identifier', { length: 100 }), // Postnummer, regionkod
  polygon: text('polygon'), // GeoJSON polygon om detaljerad geografisk data behövs
  createdAt: timestamp('created_at').defaultNow(),
});

// 9. Team-geografiska områden (koppling)
export const teamGeographicAreas = pgTable('team_geographic_areas', {
  id: uuid('id').primaryKey().defaultRandom(),
  teamId: uuid('team_id').references(() => teams.id, { onDelete: 'cascade' }).notNull(),
  geographicAreaId: uuid('geographic_area_id').references(() => geographicAreas.id, { onDelete: 'cascade' }).notNull(),
});

// Relationer
export const performersRelations = relations(performers, ({ one, many }) => ({
  user: one(users, {
    fields: [performers.userId],
    references: [users.id],
  }),
  teamMemberships: many(teamMembers),
}));

export const teamsRelations = relations(teams, ({ one, many }) => ({
  sharedUser: one(users, {
    fields: [teams.sharedUserId],
    references: [users.id],
  }),
  members: many(teamMembers),
  equipment: many(teamEquipment),
  performanceCodes: many(teamPerformanceCodes),
  geographicAreas: many(teamGeographicAreas),
}));

export const equipmentRelations = relations(equipment, ({ many }) => ({
  teams: many(teamEquipment),
}));
```

API Endpoints:**- Utförare:**

- GET /api/performers - Lista utförare
- POST /api/performers - Skapa utförare
- GET /api/performers/:id - Hämta utförare
- PUT /api/performers/:id - Uppdatera utförare
- DELETE /api/performers/:id - Ta bort utförare

• Team:

- GET /api/teams - Lista team
- POST /api/teams - Skapa team
- GET /api/teams/:id - Hämta team
- PUT /api/teams/:id - Uppdatera team
- DELETE /api/teams/:id - Ta bort team
- POST /api/teams/:id/members - Lägg till medlem
- DELETE /api/teams/:id/members/:performerId - Ta bort medlem
- POST /api/teams/:id/equipment - Lägg till utrustning
- DELETE /api/teams/:id/equipment/:equipmentId - Ta bort utrustning
- POST /api/teams/:id/performance-codes - Lägg till utförandekod
- POST /api/teams/:id/geographic-areas - Lägg till geografiskt område

• Produktionsutrustning:

- GET /api/equipment - Lista utrustning
- POST /api/equipment - Skapa utrustning
- GET /api/equipment/:id - Hämta utrustning
- PUT /api/equipment/:id - Uppdatera utrustning
- DELETE /api/equipment/:id - Ta bort utrustning

• Sidoregister:

- GET /api/performance-codes - Lista utförandekoder
- POST /api/performance-codes - Skapa utförandekod
- GET /api/geographic-areas - Lista geografiska områden
- POST /api/geographic-areas - Skapa geografiskt område

Beroenden: Ingen

GAP-202: Projekt- och Kostnadsställekodifiering

Område: Backend - Ekonomisk integrationslogik

Prioritet: ● HÖG

Beskrivning:

När en artikel klarmarkeras av utförare:

1. Kopplas till team
2. Registrerar personer och bil
3. Sätter projektkod (team/person)

4. Sätter kostnadsställe (bil)
5. Skickas till Fortnox via API

Teknisk Implementation:

Databas-schema:

```
// packages/db/src/schema/task-completion.ts
export const taskCompletions = pgTable('task_completions', {
  id: uuid('id').primaryKey().defaultRandom(),
  taskId: uuid('task_id').references(() => tasks.id, { onDelete: 'cascade' }).notNull(),
  articleId: uuid('article_id').references(() => articles.id).notNull(),

  // Team och utförare
  teamId: uuid('team_id').references(() => teams.id).notNull(),
  performerIds: jsonb('performer_ids').notNull(), // Array av utförar-ID

  // Utrustning
  equipmentId: uuid('equipment_id').references(() => equipment.id),

  // Ekonomisk kodifiering
  projectCode: varchar('project_code', { length: 100 }), // Projektkod för Fortnox
  costCenter: varchar('cost_center', { length: 100 }), // Kostnadsställe för Fortnox

  // Fakturering
  invoiced: boolean('invoiced').default(false),
  invoicedAt: timestamp('invoiced_at'),
  fortnoxInvoiceId: varchar('fortnox_invoice_id', { length: 100 }),

  completedAt: timestamp('completed_at').defaultNow(),
});
```

Backend - Kompletteringslogik:


```
// packages/api/src/services/task-completion.service.ts
export interface TaskCompletionData {
  taskId: string;
  articleId: string;
  teamId: string;
  performerIds: string[];
  equipmentId?: string;
}

export class TaskCompletionService {
  async completeTask(data: TaskCompletionData): Promise<void> {
    // 1. Hämta team-information
    const team = await db.query.teams.findFirst({
      where: eq(teams.id, data.teamId),
    });

    if (!team) throw new Error('Team not found');

    // 2. Hämta utrustning (om angiven)
    let costCenter = null;
    if (data.equipmentId) {
      const equipment = await db.query.equipment.findFirst({
        where: eq(equipment.id, data.equipmentId),
      });
      costCenter = equipment?.costCenter;
    }

    // 3. Generera projektkod (baserat på team)
    const projectCode = `TEAM-${team.id.substring(0, 8)}`;

    // 4. Spara komplettering
    const completion = await db.insert(taskCompletions).values({
      taskId: data.taskId,
      articleId: data.articleId,
      teamId: data.teamId,
      performerIds: JSON.stringify(data.performerIds),
      equipmentId: data.equipmentId,
      projectCode,
      costCenter,
    }).returning();

    // 5. Skicka till Fortnox (om konfigurerat)
    await this.sendToFortnox(completion[0]);

    // 6. Uppdatera uppgiftsstatus
    await db.update(tasks)
      .set({ status: 'completed', completedAt: new Date() })
      .where(eq(tasks.id, data.taskId));
  }

  private async sendToFortnox(completion: TaskCompletion): Promise<void> {
    // Implementera Fortnox API-anrop
    // POST till Fortnox med artikel, projektkod och kostnadsställe

    // Exempel (pseudo-kod):
    // const fortnoxApi = new FortnoxAPI();
    // await fortnoxApi.createInvoiceRow({
    //   articleNumber: completion.articleId,
    //   projectCode: completion.projectCode,
    //   costCenter: completion.costCenter,
    //   quantity: ...,
    //   price: ...,
  }

```

```
    // });  
  }  
}
```

API Endpoints:

- POST /api/tasks/:id/complete - Slutför uppgift med team/utrustning
- GET /api/task-completions/:id - Hämta kompletteringsdetaljer
- POST /api/task-completions/:id/invoice - Skicka till Fortnox (manuell trigger)

Beroenden: GAP-201

GAP-203: Leverantörsregister och Artikelkoppling

Område: Backend - Datamodell

Prioritet: 🟡 MEDIUM

Beskrivning:

- Sidoregister för leverantörer
- Artiklar kan ha flera leverantörer
- Leverantörens artikelnummer kopplas till varje leverantör
- Möjlig rangordning (prioritet)

Teknisk Implementation:**Databas-schema:**

```
// packages/db/src/schema/suppliers.ts

// 1. Leverantörsregister
export const suppliers = pgTable('suppliers', {
  id: uuid('id').primaryKey().defaultRandom(),
  name: varchar('name', { length: 255 }).notNull(),
  organizationNumber: varchar('organization_number', { length: 50 }),
  email: varchar('email', { length: 255 }),
  phone: varchar('phone', { length: 50 }),
  address: text('address'),
  website: varchar('website', { length: 255 }),
  isActive: boolean('is_active').default(true),
  createdAt: timestamp('created_at').defaultNow(),
  updatedAt: timestamp('updated_at').defaultNow(),
});

// 2. Artikel-leverantör-koppling
export const articleSuppliers = pgTable('article_suppliers', {
  id: uuid('id').primaryKey().defaultRandom(),
  articleId: uuid('article_id').references(() => articles.id, { onDelete: 'cascade' })
    .notNull(),
  supplierId: uuid('supplier_id').references(() => suppliers.id, { onDelete:
    'cascade' }).notNull(),

  supplierArticleNumber: varchar('supplier_article_number', { length: 100 }), //
  // Leverantörens artikelnummer
  purchasePrice: decimal('purchase_price', { precision: 10, scale:
    2 }), // Inköpspris från denna leverantör
  leadTimeDays: integer('lead_time_days'), // Leveranstid i dagar

  priority: integer('priority').default(0), // Lägre = högre prioritet (0 = första
  // valet)
  isPreferred: boolean('is_preferred').default(false),

  createdAt: timestamp('created_at').defaultNow(),
  updatedAt: timestamp('updated_at').defaultNow(),
});

// Relationer
export const suppliersRelations = relations(suppliers, ({ many }) => ({
  articles: many(articleSuppliers),
}));

export const articleSuppliersRelations = relations(articleSuppliers, ({ one }) => ({
  article: one(articles, {
    fields: [articleSuppliers.articleId],
    references: [articles.id],
  }),
  supplier: one(suppliers, {
    fields: [articleSuppliers.supplierId],
    references: [suppliers.id],
  }),
}));
```

API Endpoints:

- GET /api/suppliers - Lista leverantörer
- POST /api/suppliers - Skapa leverantör
- GET /api/suppliers/:id - Hämta leverantör
- PUT /api/suppliers/:id - Uppdatera leverantör
- DELETE /api/suppliers/:id - Ta bort leverantör

- GET /api/articles/:id/suppliers - Hämta artikelns leverantörer
- POST /api/articles/:id/suppliers - Lägg till leverantör till artikel
- PUT /api/articles/:id/suppliers/:supplierId - Uppdatera leverantörs-relation
- DELETE /api/articles/:id/suppliers/:supplierId - Ta bort leverantör från artikel

Beroenden: Ingen

3.3 MEDIUM GAP (Förbättringar och användarupplevelse)

GAP-301: Sidoregister - Artikeltyper och Utförandekoder

Område: Backend - Datamodell

Prioritet: 🟡 MEDIUM

Beskrivning:

Enkla sidoregister för:

- Artikeltyper (vara, tjänst, kontroll)
- Utförandekoder (för artiklar och att-göra-listor)
- Ikonregister

Teknisk Implementation:

Databas-schema:

```
// packages/db/src/schema/registries.ts

// 1. Artikeltyper
export const articleTypes = pgTable('article_types', {
  id: uuid('id').primaryKey().defaultRandom(),
  name: varchar('name', { length: 100 }).notNull().unique(),
  description: text('description'),
  isActive: boolean('is_active').default(true),
  createdAt: timestamp('created_at').defaultNow(),
});

// 2. Utförandekoder (används redan i GAP-201, men listning här för helhet)
// export const performanceCodes = ... (se GAP-201)

// 3. Ikonregister
export const icons = pgTable('icons', {
  id: uuid('id').primaryKey().defaultRandom(),
  name: varchar('name', { length: 100 }).notNull(),
  url: varchar('url', { length: 500 }).notNull(), // URL till bildfil
  category: varchar('category', { length: 100 }), // Gruppering av ikoner
  isActive: boolean('is_active').default(true),
  createdAt: timestamp('created_at').defaultNow(),
});

// Uppdatera articles-schema för att referera till articleTypes
// packages/db/src/schema/articles.ts
export const articles = pgTable('articles', {
  // ... befintliga fält ...
  articleTypeId: uuid('article_type_id').references(() => articleTypes.id),
  iconId: uuid('icon_id').references(() => icons.id),
  // ... övriga fält ...
});
```

API Endpoints:

- GET /api/article-types - Lista artikeltyper
- POST /api/article-types - Skapa artikeltyp
- GET /api/icons - Lista ikoner
- POST /api/icons - Skapa ikon (med uppladdning)

Beroenden: Ingen

GAP-302: Metadata Sorteringsordning - Buggfix

Område: Backend - Buggfix

Prioritet: 🟡 MEDIUM

Beskrivning:

Sorteringsordning för metadatafält fungerar inte korrekt.

Teknisk Implementation:**Investigation och Fix:**

```
// packages/api/src/services/metadata.service.ts

// Nuvarande problem: Sorteringsordning respekteras inte i queries
// Lösning: Säkerställ att sortOrder används konsekvent

export class MetadataService {
  async getMetadataFields(options?: {
    includeInactive?: boolean;
    groupBy?: string;
  }): Promise<MetadataField[]> {
    let query = db.query.metadataFields.findMany({
      where: options?.includeInactive
        ? undefined
        : eq(metadataFields.isActive, true),
      // KRITISK: Sortera alltid efter sortOrder
      orderBy: [
        asc(metadataFields.sortOrder),
        asc(metadataFields.name), // Sekundär sortering
      ],
    });

    return query;
  }

  async updateSortOrder(fieldId: string, newOrder: number): Promise<void> {
    // Validera att newOrder är unikt (eller hantera konflikter)
    await db.update(metadataFields)
      .set({
        sortOrder: newOrder,
        updatedAt: new Date(),
      })
      .where(eq(metadataFields.id, fieldId));
  }

  // Batch-uppdatering av sorteringsordning
  async reorderFields(orderedFieldIds: string[]): Promise<void> {
    await db.transaction(async (tx) => {
      for (let i = 0; i < orderedFieldIds.length; i++) {
        await tx.update(metadataFields)
          .set({ sortOrder: i })
          .where(eq(metadataFields.id, orderedFieldIds[i]));
      }
    });
  }
}
```

Test:

```
// packages/api/src/services/__tests__/metadata.service.test.ts
describe('MetadataService - Sort Order', () => {
  it('should return fields in correct sort order', async () => {
    // Skapa testfält med olika sortOrder
    await createMetadataField({ name: 'Titel', sortOrder: 3 });
    await createMetadataField({ name: 'Följande', sortOrder: 4 });
    await createMetadataField({ name: 'E-post', sortOrder: 5 });

    const fields = await metadataService.getMetadataFields();

    expect(fields[0].name).toBe('Titel');
    expect(fields[1].name).toBe('Följande');
    expect(fields[2].name).toBe('E-post');
  });
});
```

Beroenden: GAP-101

GAP-303: Menyförenkling och Inställningsdolda Funktioner

Område: Frontend

Prioritet: ● LÅG

Beskrivning:

Förenkla menyer och dölj oanvända funktioner via inställningar.

Huvudfunktioner som ska vara synliga:

1. Tanka in och uppdatera objekt
2. Skapa orderkoncept
3. Skapa och hantera artiklar
4. Hantera metadata
5. Lägga upp separata prislistor för kunder
6. (Utförartidsregister per utförare)

Teknisk Implementation:

Frontend - Meny-konfiguration:

```
// apps/mobile/src/config/menu-config.ts
export interface MenuItem {
  id: string;
  label: string;
  route: string;
  icon: string;
  visible: boolean; // Kan döljas via inställningar
  category: 'core' | 'advanced' | 'admin';
}

export const DEFAULT_MENU_ITEMS: MenuItem[] = [
  // CORE - Alltid synliga
  { id: 'objects', label: 'Objekt', route: '/objects', icon: 'cube', visible: true, category: 'core' },
  { id: 'order-concepts', label: 'Orderkoncept', route: '/order-concepts', icon: 'document-text', visible: true, category: 'core' },
  { id: 'articles', label: 'Artiklar', route: '/articles', icon: 'pricetag', visible: true, category: 'core' },
  { id: 'metadata', label: 'Metadata', route: '/metadata', icon: 'settings', visible: true, category: 'core' },
  { id: 'pricelists', label: 'Prislistor', route: '/pricelists', icon: 'cash', visible: true, category: 'core' },

  // ADVANCED - Kan döljas
  { id: 'performer-time', label: 'Utförartider', route: '/performer-time', icon: 'time', visible: false, category: 'advanced' },
  { id: 'production-timeline', label: 'Produktionstidslista', route: '/production-timeline', icon: 'calendar', visible: false, category: 'advanced' },
];

// Inställningar för att visa/dölja menyalternativ
export const MenuSettingsScreen = () => {
  const [menuItems, setMenuItems] = useState(DEFAULT_MENU_ITEMS);

  const toggleVisibility = (id: string) => {
    setMenuItems(items =>
      items.map(item =>
        item.id === id ? { ...item, visible: !item.visible } : item
      )
    );
  };

  return (
    <View>
      <Text style={{ fontSize: 18, fontWeight: 'bold', marginBottom: 10 }}>
        Menyinställningar
      </Text>
      {menuItems.filter(item => item.category !== 'core').map(item => (
        <View key={item.id} style={{ flexDirection: 'row', alignItems: 'center', marginBottom: 10 }}>
          <Switch
            value={item.visible}
            onChange={() => toggleVisibility(item.id)}
          />
          <Text style={{ marginLeft: 10 }}>{item.label}</Text>
        </View>
      ))}
    </View>
  );
};
```


Beroenden: Ingen

4. Uppdaterad Implementationsroadmap

4.1 Sprint 1: Metadata & Villkor (Vecka 1-2)

Mål: Grundläggande metadatahantering och villkorstestning

Task	GAP	Prioritet	Estimat	Beroenden
Implementera hierarkisk metadatastruktur	GAP-101	 KRITISK	3d	-
Skapa villkorsmotor och testfunktion	GAP-102	 KRITISK	4d	GAP-101
Implementera standardiserad matchningsfunktion	GAP-103	 KRITISK	3d	GAP-101
Ta bort "Objekttyp"-funktion	GAP-103	 KRITISK	0.5d	GAP-103
Fixa metadata-sorteringsordning	GAP-302	 MEDIUM	1d	GAP-101

Deliverables:

-  Metadata med parent-child struktur
-  Sammansatta namn (t.ex. "kontaktpost")
-  Testfunktion för villkor
-  Enhetlig filterkomponent
-  Fungerande sortering

4.2 Sprint 2: Artikelhantering - Ekonomi & Metadata (Vecka 3-4)

Mål: Komplett artikelhantering med prisberäkning och metadata-koppling

Task	GAP	Prioritet	Estimat	Beroenden
Implementera artikelprisberäkning	GAP-104	● HÖG	2d	-
Skapa artikel-metadata-bindningar	GAP-105	● HÖG	3d	GAP-101
Fast antal och dold antalsinmatning	GAP-106	● MEDIUM	1d	-
Strukturartiklar (underartiklar)	GAP-107	● MEDIUM	2d	-
Leverantörsregister och koppling	GAP-203	● MEDIUM	2d	-

Deliverables:

- ☒ Automatisk prisberäkning (inköp → frakt → lager → påslag → listpris)
- ☒ Artiklar kan hämta och visa metadata från objekt
- ☒ Tre beteenden: visa, tillåt uppdatering, krav
- ☒ Fast antal-logik för besiktningsartiklar
- ☒ Strukturartiklar med under-artiklar
- ☒ Leverantörshantering

4.3 Sprint 3: Team & Produktionsutrustning (Vecka 5-6)

Mål: Komplet team- och utrustningshantering

Task	GAP	Prioritet	Estimat	Beroenden
Skapa utförare-register	GAP-201	● HÖG	2d	-
Skapa team-register	GAP-201	● HÖG	2d	-
Skapa produktionsutrustnings-register	GAP-201	● HÖG	2d	-
Implementera kopplingar (team-utförare-utrustning)	GAP-201	● HÖG	3d	-
Utförandekoder och geografiska områden	GAP-201	● HÖG	2d	-
Sidoregister (artikeltyper, ikoner)	GAP-301	● MEDIUM	1d	-

Deliverables:

- ✓ Register för utförare, team, produktionsutrustning
- ✓ Team-medlemskap och utrustningskopplingar
- ✓ Utförandekoder per team
- ✓ Geografiska områden per team
- ✓ Sidoregister för artikeltyper och ikoner

4.4 Sprint 4: Projekt- & Kostnadsställekodifiering + Fortnox (Vecka 7-8)

Mål: Ekonomisk uppföljning och Fortnox-integration

Task	GAP	Prioritet	Estimat	Beroenden
Implementera task completion med team-koppling	GAP-202	● HÖG	3d	GAP-201
Projektkod- och kostnadsställe-generering	GAP-202	● HÖG	2d	GAP-201, GAP-202
Fortnox API-integration	GAP-202	● HÖG	4d	GAP-202
Ekonomisk uppföljning (rapporter)	GAP-202	● MEDIUM	3d	GAP-202

Deliverables:

- ✓ Artikel-komplettering registrerar team, personer, bil
- ✓ Automatisk projekt- och kostnadsställekodifiering
- ✓ Fortnox API-anrop vid artikelkomplettering
- ✓ Rapporter för ekonomisk uppföljning

4.5 Sprint 5: UI/UX-förbättringar (Vecka 9)

Mål: Förenkla gränssnitt och förbättra användarupplevelse

Task	GAP	Prioritet	Estimat	Beroenden
Menyförenkling och inställningar	GAP-303	● LÅG	2d	-
UI-tester och bugfixar	-	● LÅG	3d	Alla tidigare
Dokumentation och user guides	-	● LÅG	2d	Alla tidigare

Deliverables:

- ✓ Förenklad huvudmeny
- ✓ Inställningar för att visa/dölja funktioner
- ✓ Buggfria UI-komponenter
- ✓ Användardokumentation

5. Teknisk Stack & Arkitektur

5.1 Frontend (React Native/Expo)

```
apps/mobile/
├── src/
│   ├── components/
│   │   ├── conditions/
│   │   │   └── ConditionTester.tsx (GAP-102)
│   │   ├── filters/
│   │   │   └── MetadataFilter.tsx (GAP-103)
│   │   ├── tasks/
│   │   │   ├── TaskMetadataFields.tsx (GAP-105)
│   │   │   └── TaskQuantityInput.tsx (GAP-106)
│   │   └── articles/
│   │       └── ArticlePricingForm.tsx (GAP-104)
│   ├── screens/
│   │   ├── metadata/
│   │   ├── articles/
│   │   ├── teams/
│   │   └── settings/
│   └── config/
│       └── menu-config.ts (GAP-303)
```

5.2 Backend (Express + Drizzle ORM)

```
packages/api/
├── src/
│   ├── services/
│   │   ├── condition-engine.ts (GAP-102)
│   │   ├── filter.service.ts (GAP-103)
│   │   ├── metadata.service.ts (GAP-101, GAP-302)
│   │   ├── article-pricing.service.ts (GAP-104)
│   │   ├── article-metadata.service.ts (GAP-105)
│   │   ├── article-structure.service.ts (GAP-107)
│   │   ├── task-completion.service.ts (GAP-202)
│   │   └── fortnox.service.ts (GAP-202)
│   └── routes/
│       ├── metadata.ts
│       ├── articles.ts
│       ├── conditions.ts
│       ├── teams.ts
│       ├── performers.ts
│       └── equipment.ts
└── packages/db/
    ├── src/
    │   ├── schema/
    │   │   ├── metadata.ts (GAP-101)
    │   │   ├── conditions.ts (GAP-102)
    │   │   ├── articles.ts (GAP-104, GAP-106, GAP-107)
    │   │   ├── team.ts (GAP-201)
    │   │   ├── suppliers.ts (GAP-203)
    │   │   ├── task-completion.ts (GAP-202)
    │   │   └── registries.ts (GAP-301)
    │   └── migrations/
    │       └── ... (alla GAP-relaterade migreringar)
```

6. API Endpoints Sammanfattning

6.1 Metadata

- GET /api/metadata/fields?includeHierarchy=true
- GET /api/metadata/fields/:id/hierarchy
- PATCH /api/metadata/fields/:id/sort-order
- POST /api/metadata/fields/:id/children

6.2 Villkor & Testning

- POST /api/conditions/test
- GET /api/conditions/:id/evaluate
- POST /api/articles/:id/conditions/test
- POST /api/order-concepts/:id/conditions/test

6.3 Filtrering

- POST /api/articles/filter
- POST /api/objects/filter
- POST /api/order-concepts/filter

6.4 Artiklar

- PUT /api/articles/:id/pricing
- POST /api/articles/pricing/calculate
- GET /api/articles/:id/metadata-bindings
- POST /api/articles/:id/metadata-bindings
- GET /api/articles/:id/metadata/objectId
- POST /api/articles/:id/metadata/objectId/validate
- GET /api/articles/:id/children
- POST /api/articles/:id/children
- GET /api/articles/:id/suppliers
- POST /api/articles/:id/suppliers

6.5 Team & Utförare

- GET /api/performers
- POST /api/performers
- GET /api/teams
- POST /api/teams
- POST /api/teams/:id/members
- POST /api/teams/:id/equipment
- POST /api/teams/:id/performance-codes
- POST /api/teams/:id/geographic-areas

6.6 Produktionsutrustning

- GET /api/equipment
- POST /api/equipment
- GET /api/equipment/:id

6.7 Uppgiftskomplettering

- `POST /api/tasks/:id/complete`
- `GET /api/task-completions/:id`
- `POST /api/task-completions/:id/invoice`

6.8 Sidoregister

- `GET /api/article-types`
- `POST /api/article-types`
- `GET /api/performance-codes`
- `POST /api/performance-codes`
- `GET /api/geographic-areas`
- `POST /api/geographic-areas`
- `GET /api/icons`
- `POST /api/icons`
- `GET /api/suppliers`
- `POST /api/suppliers`

7. Prioriterad To-Do Lista (Nästa Steg)

Prioritet 1: Starta NU (Kritiska blockerare)

1. **GAP-101:** Implementera hierarkisk metadatastruktur
 - Skapa `metadata_fields` schema med `parentId`
 - Implementera logik för sammansatta namn
 - API endpoints för hierarki-navigering
2. **GAP-102:** Villkorstestning och validering
 - Skapa `conditions` och relationsscheman
 - Implementera `ConditionEngine` med evaluering
 - Frontend-komponent för testning
3. **GAP-103:** Standardiserad matchningsfunktion
 - Skapa återanvändbar `MetadataFilter` komponent
 - Implementera `FilterService` i backend
 - Migrera bort "Objekttyp"-funktionen

Prioritet 2: Vecka 1-2 (Höga GAP)

1. **GAP-104:** Artikelprisberäkning
 - Utöka `articles` schema med ekonomifält
 - Implementera `ArticlePricingService`
 - Frontend-formulär med automatisk beräkning
2. **GAP-105:** Artikel-metadata-bindningar
 - Skapa `article_metadata_bindings` schema
 - Implementera `ArticleMetadataService`
 - Frontend-komponent för metadata-visning i uppgifter

Prioritet 3: Vecka 3-4 (Team-system)

1. **GAP-201:** Team, utförare och produktionsutrustning
 - Skapa alla team-relaterade scheman
 - Implementera API endpoints
 - Frontend-register och kopplingsformulär
2. **GAP-202:** Projekt- och kostnadsställekodifiering
 - Skapa `task_completions` schema
 - Implementera `TaskCompletionService`
 - Fortnox API-integration

Prioritet 4: Vecka 5+ (Nice-to-have)

1. **GAP-106:** Fast antal och dold antalsinmatning
2. **GAP-107:** Strukturartiklar
3. **GAP-203:** Leverantörsregister
4. **GAP-301:** Sidoregister (artikeltyper, ikoner)
5. **GAP-302:** Fixa metadata-sortering
6. **GAP-303:** Menyförenkling

8. Risker och Beroenden

8.1 Tekniska Risker

Risk	Sannolikhet	Impact	Mitigation
Fortnox API-kompatibilitet	Medel	Hög	Tidig prototyp och tester mot Fortnox sandbox
Metadata-hierarki prestanda	Låg	Medel	Indexering och caching av hierarkiska queries
Villkorsmotorns komplexitet	Medel	Hög	Börja med enkla operatorer, utöka gradvis
Team/utrustning-kopplingar	Låg	Medel	Tydlig datamodell och relationer

8.2 Kritiska Beroenden

- **GAP-101** är beroende för: GAP-102, GAP-103, GAP-105, GAP-302
- **GAP-201** är beroende för: GAP-202
- **GAP-102** är beroende för: GAP-103

8.3 Integrationspunkter

- **Fortnox API:** OAuth, fakturor, projekt, kostnadsställen
- **Befintlig orderhantering:** 4-steps flöde måste integreras med nya team/utrustning-kopplingar
- **Befintlig metadata:** Migration från nuvarande struktur till hierarkisk

9. Mätbara Framgångskriterier

9.1 Sprint 1 (Metadata & Villkor)

- ✓ Alla metadatafält visas med korrekta, sammansatta namn
- ✓ Sorteringsordning fungerar korrekt
- ✓ Testfunktion för villkor ger korrekta resultat
- ✓ Standardiserad filterkomponent används på minst 3 platser
- ✓ "Objekttyp" är borttagen

9.2 Sprint 2 (Artiklar)

- ✓ Listpris beräknas automatiskt från inköpspris + kostnader + påslag
- ✓ Marginalprocent och marginal per enhet visas korrekt
- ✓ Artiklar kan hämta och visa metadata från objekt
- ✓ Besiktningsartiklar visar inget antalsfält i appen
- ✓ Leverantörer kan kopplas till artiklar med egna artikelnummer

9.3 Sprint 3 (Team)

- ✓ Utförare kan registreras och kopplas till användarkonton
- ✓ Team kan skapas med flera utförare
- ✓ Produktionsutrustning (bilar) kan kopplas till team
- ✓ Utförandekoder och geografiska områden kan kopplas till team

9.4 Sprint 4 (Ekonomi)

- ✓ Vid artikelkomplettering registreras team, personer och bil
- ✓ Projektkod och kostnadsställe genereras automatiskt
- ✓ Data skickas till Fortnox vid fakturering
- ✓ Ekonomisk uppföljning visar kostnader per bil och team

10. Nästa Konkreta Steg i Replit

Steg 1: Skapa Metadata-schema (GAP-101)

```
cd packages/db
# Skapa ny fil: src/schema/metadata.ts
# Kör migration: pnpm db:migrate
```

Steg 2: Implementera Villkorsmotor (GAP-102)

```
cd packages/api
# Skapa ny fil: src/services/condition-engine.ts
# Skapa routes: src/routes/conditions.ts
```

Steg 3: Skapa Filterkomponent (GAP-103)

```
cd apps/mobile
# Skapa ny fil: src/components/filters/MetadataFilter.tsx
# Testa i artikelskärm
```

Steg 4: Fortsätt enligt Roadmap

Följ Sprint 1-5 i sektion 4 för strukturerad implementation.

Slutsats

Session 2026-06-16T17-42 har avsevärt förtydligat och utökat kravbilden för Traivo-systemet. De mest kritiska GAP är:

1. **GAP-101**: Hierarkisk metadatastruktur
2. **GAP-102**: Villkorstestning
3. **GAP-103**: Standardiserad matchningsfunktion
4. **GAP-201**: Team & produktionsutrustning
5. **GAP-202**: Projekt- och kostnadsställekodifiering

Denna gap-analys ger en komplett, teknisk roadmap för implementation i Replit med konkreta datamodeller, API-endpoints och komponenter. Följ den prioriterade to-do-listan för att systematiskt stänga GAP:en och nå full funktionalitet.

Estimerad total implementationstid: 9 veckor (5 sprints)

Kritisk väg: GAP-101 → GAP-102 → GAP-103 → GAP-201 → GAP-202

Dokumentet uppdateras vid varje ny session eller GAP-upptäckt.